计算机系统基础实验

PA2

姓名: **伍知行**　学号: **251220081**

November 2, 2026

1　实验进度

我完成的实验内容如下:

1. 实现 RV32I 指令集中绝大部分的指令处理, 能够运行所有本地 cpu-tests;
2. 完善基础设施, 包括指令环形缓存区、mtrace、ftrace 等;
3. 实现 klib 中的大量库函数并进行测试;
4. 完成 IOE 部分, 实现冯诺依曼计算机系统。

2　必答题

I. 理解 YEMU 如何执行程序:

- YEMU 上执行加法程序的状态机:
  用一个 5 元组 (R[0],R[1],M[x],M[y],M[z]) 表示加法程序每一步指令执行之后各寄存器/内存存储的数据:

$$
\begin{aligned}
\text{初始}(0,0,16,33,0) &\xrightarrow{\text{第一步}} (33,0,16,33,0)\\
&\xrightarrow{\text{第二步}} (33,33,16,33,0)\\
&\xrightarrow{\text{第三步}} (16,33,16,33,0)\\
&\xrightarrow{\text{第四步}} (49,33,16,33,0)\\
&\xrightarrow{\text{第五步}} (49,33,16,33,49)
\end{aligned}
$$

- 理解 YEMU 如何执行一条指令:

首先根据 pc 从存储器中取出当前指令, 然后根据指令码 op 进行相对应的操作数译码与指令的执行，且在遇到非法指令码时中断，最后 pc 自增指向下一条指令。

- 以上两者的联系是，执行整个加法程序的过程是由每一步指令的执行过程共同构成的，且单步指令的执行过程也可以视作状态机。

II. RTFSC 理解指令执行的过程:

以 RISCV 指令集中的指令 `auipc` 为例, 它在 nemu 中的执行过程如下:

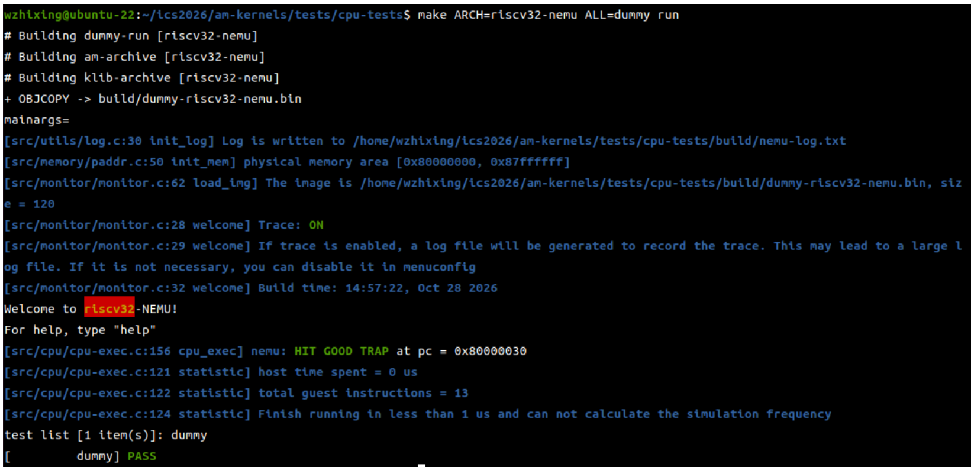
1. 首先, 在 nemu/src/isa/riscv32/inst.c 中的 `isa_exec_once` 中, 指令内容被 `inst_fetch` 函数取出, 并存放到一个 Decode 类型的结构体的 isa.inst 部分;
2. 接下来, 调用 `decode_exec` 函数, 进行取指以后的部分;
3. 在 `decode_exec` 函数中, 宏 `INSTPAT` 定义了一种匹配方式, 它模拟了根据操作码 opcode、三位功能码 funct3 和七位功能码 funct7 来确定指令含义的过程。特别地, 对于 `auipc` 而言, 用于匹配的 32 位串是"??????? ????? ????? ??? ????? 00101 11", 指令匹配成功后进入解码操作数的步骤;
4. 对于 `auipc` 而言, 它是 U 型指令, 它没有源寄存器, 因此只需读取出立即数 imm_U, 也就是 32 位指令中的高 20 位;
5. 接着执行 `auipc` 的指令内容, 即 R[rd]$\leftarrow$ pc + (imm_U $\ll$ 12)。因为该指令不涉及访存, 所以直接将结果写回寄存器 rd。
6. 最后根据 RISCV 的约定, 将 `x0` 寄存器的值赋值为 0, 整个指令执行结束。

III. 准备交叉编译环境:

根据手册中的要求下载相应的工具: `g++-riscv64-linux-gnu` 和 `binutils-riscv64-linux-gnu` 即可。

IV. 运行第一个客户程序:

实现完成大部分 RISCV 指令集中的指令后, 在 nemu 中运行 dummy:



输出结果为 “HIT GOOD TRAP”, 表明客户程序正常运行并退出 nemu。

V. 实现更多的指令:

在 nemu/src/isa/riscv32/inst.c 的 `decode_exec` 中补充更多的 `INSTPAT`, 使得能够处理更多的 RISCV 指令。

```
INSTPAT("??????? ????? ????? 010 ????? 00000 11", lw     , I, R(rd) = Mr(src1 + imm, 4));
INSTPAT("??????? ????? ????? 100 ????? 00000 11", lbu    , I, R(rd) = Mr(src1 + imm, 1));
INSTPAT("??????? ????? ????? 101 ????? 00000 11", lhu    , I, R(rd) = Mr(src1 + imm, 2));
INSTPAT("??????? ????? ????? 000 ????? 01000 11", sb     , S, Mw(src1 + imm, 1, src2));
INSTPAT("??????? ????? ????? 001 ????? 01000 11", sh     , S, Mw(src1 + imm, 2, src2));
INSTPAT("??????? ????? ????? 010 ????? 01000 11", sw     , S, Mw(src1 + imm, 4, src2));

INSTPAT("??????? ????? ????? 000 ????? 00100 11", addi   , I, R(rd) = src1 + imm);
INSTPAT("??????? ????? ????? 010 ????? 00100 11", slti   , I, R(rd) = (int32_t)src1 < (int32_t)imm);
INSTPAT("??????? ????? ????? 011 ????? 00100 11", sltiu  , I, R(rd) = src1 < imm);
INSTPAT("0000000 ????? ????? 001 ????? 00100 11", slli   , I, R(rd) = src1 << (imm & 0x1f));
INSTPAT("0000000 ????? ????? 101 ????? 00100 11", srli   , I, R(rd) = src1 >> (imm & 0x1f));
INSTPAT("0100000 ????? ????? 101 ????? 00100 11", srai   , I, R(rd) = ((int32_t)src1 >> (imm & 0x1f)));
INSTPAT("??????? ????? ????? 100 ????? 00100 11", xori   , I, R(rd) = src1 ^ imm);
INSTPAT("??????? ????? ????? 110 ????? 00100 11", ori    , I, R(rd) = src1 | imm);
INSTPAT("??????? ????? ????? 111 ????? 00100 11", andi   , I, R(rd) = src1 & imm);

INSTPAT("0000000 ????? ????? 000 ????? 01100 11", add    , R, R(rd) = src1 + src2);
INSTPAT("0100000 ????? ????? 000 ????? 01100 11", sub    , R, R(rd) = src1 - src2);
INSTPAT("0000000 ????? ????? 001 ????? 01100 11", sll    , R, R(rd) = src1 << (src2 & 0x1f));
INSTPAT("0000000 ????? ????? 010 ????? 01100 11", slt    , R, R(rd) = (int32_t)src1 < (int32_t)src2);
INSTPAT("0000000 ????? ????? 011 ????? 01100 11", sltu   , R, R(rd) = src1 < src2);
INSTPAT("0000000 ????? ????? 100 ????? 01100 11", xor    , R, R(rd) = src1 ^ src2);
INSTPAT("0000000 ????? ????? 101 ????? 01100 11", srl    , R, R(rd) = src1 >> (src2 & 0x1f));
INSTPAT("0100000 ????? ????? 101 ????? 01100 11", sra    , R, R(rd) = ((int32_t)src1 >> (src2 & 0x1f)));
INSTPAT("0000000 ????? ????? 110 ????? 01100 11", or     , R, R(rd) = src1 | src2);
INSTPAT("0000000 ????? ????? 111 ????? 01100 11", and    , R, R(rd) = src1 & src2);
```

图中展示了部分已实现的指令。

接着在 cpu-tests 目录下运行其他测试程序:

```
test list [33 item(s)]: fact div leap-year shuixianhua wanshu load-store if-else mov-c recursion add shift movsx min3 mul-longlong t
o-lower-case unalign fib select-sort bubble-sort prime sub-longlong bit add-longlong pascal max matrix-mul mersenne goldbach crc32 d
ummy sum quick-sort switch
[          fact] PASS
[           div] PASS
[     leap-year] PASS
[   shuixianhua] PASS
[        wanshu] PASS
[    load-store] PASS
[       if-else] PASS
[         mov-c] PASS
[     recursion] PASS
[           add] PASS
[         shift] PASS
[         movsx] PASS
[          min3] PASS
[  mul-longlong] PASS
[ to-lower-case] PASS
[       unalign] PASS
[           fib] PASS
[   select-sort] PASS
[   bubble-sort] PASS
[         prime] PASS
[  sub-longlong] PASS
[           bit] PASS
[  add-longlong] PASS
[        pascal] PASS
[           max] PASS
[    matrix-mul] PASS
[      mersenne] PASS
[      goldbach] PASS
[         crc32] PASS
[         dummy] PASS
[           sum] PASS
[    quick-sort] PASS
[        switch] PASS
```

可以发现能够成功通过除 string 和 hello-str 外其他测试程序。

VI. 阅读 Makefile:

abstract-machine 项目的 Makefile 设计如下:

a. 第一部分: 基础设置和检查。这一部分主要是做好基础准备，并核查包括 AM 和架构 (ARCH) 在内的环境变量, 以及检查是否有合法的源对象 (SRCS)。

b. 第二部分: 通用的编译目标。在这个部分, Makefile 创建了 build 目录 (也就是目的文件夹)、定义了镜像文件和库 (archive) 并汇总所有需要链接的文件 (.o 和.a)。

c. 第三部分: 通用的编译标志。这一部分定义了 (交叉) 编译器以及编译时需要的标志, 如 CFLAGS 和 INC_FLAGS 等。

d. 第四部分: 架构中特有的配置。这一部分把我们 make 时所选择的架构 ARCH 中特有的配置包含、拷贝至本 Makefile 中。

e. 第五部分: 编译规则。这一部分定义了各类文件在各类编译器下编译成目标文件 (.o) 或链接库 (.a) 的规则，以及最终将所有文件和库链接起来生成最终的 elf 文件的规则。

f. 第六部分: 杂项。这一部分定义了一些非编译、链接相关的 make 命令，包括 run、clean 等。

VII. 通过批处理模式运行 NEMU:

首先阅读 nemu/src/monitor/monitor.c 中的 `parse_args` 函数, 发现批处理模式的启动依赖于参数 “-b” 或 “--batch”。

接下来在 abstract-machine 项目下寻找进入 nemu 目录相关的 Makefile/.mk 文件, 最终在 abstract-machine/scripts/platform/nemu.mk 中找到 “make run” 命令对应的实际行为:

```
run: insert-arg
    $(MAKE) -C $(NEMU_HOME) ISA=$(ISA) run ARGS="$(NEMUFLAGS)" IMG=$(IMAGE).bin
```

因此在该文件中的 NEMUFLAGS 中添加上 “-b” 便可实现批处理模式运行 nemu。

VIII. 实现字符串处理函数:

实现 abstract-machine/klib/src/string.c 中列出的字符串处理函数, 包括 `strlen`, `strcpy` 等。

```c
size_t strlen(const char *s) {
  // panic("Not implemented");
  size_t len = 0;
  while(s[len] != '\0') len ++;
  return len;
}

char *strcpy(char *dst, const char *src) {
  // panic("Not implemented");
  size_t i, len = strlen(src);
  for (i = 0; i < len; i ++) {
    dst[i] = src[i];
  }
  dst[len] = '\0';
  return dst;
}

char *strncpy(char *dst, const char *src, size_t n) {
  // panic("Not implemented");
  size_t i;
  for (i = 0; i < n && src[i] != '\0'; i ++) {
    dst[i] = src[i];
  }
  for (; i < n; i ++) {
    dst[i] = '\0';
  }
  return dst;
}

char *strcat(char *dst, const char *src) {
  // panic("Not implemented");
  size_t i, len_dst = strlen(dst), len_src = strlen(src);
  for (i = 0; i < len_src; i ++) {
    dst[len_dst + i] = src[i];
  }
  dst[len_dst + len_src] = '\0';
  return dst;
}

int strcmp(const char *s1, const char *s2) {
  // panic("Not implemented");
  size_t i, len1 = strlen(s1), len2 = strlen(s2);
  size_t max_len = len1 <= len2 ? len2 : len1;
  for (i = 0; i < max_len; i ++) {
    if (s1[i] != s2[i]) return s1[i] - s2[i];
  }
  return 0;
}

int strncmp(const char *s1, const char *s2, size_t n) {
  // panic("Not implemented");
  size_t i;
  for (i = 0; i < n; i ++) {
    if (s1[i] != s2[i]) return s1[i] - s2[i];
    if (s1[i] == '\0' || s2[i] == '\0') return 0; //Avoid overflow
  }
  return 0;
}

void *memset(void *s, int c, size_t n) {
  // panic("Not implemented");
  size_t i = 0;
  unsigned char *p = (unsigned char *)s;
  for (i = 0; i < n; i ++) {
    p[i] = c & 0xff;
  }
  return s;
}
```

```c
void *memmove(void *dst, const void *src, size_t n) {
  // panic("Not implemented");
  size_t i;
  unsigned char *p_src = (unsigned char *)src, *p_dst = (unsigned char *)dst;
  if (p_src > p_dst) {
    for (i = 0; i < n; i ++) {
      p_dst[i] = p_src[i];
    }
  }
  else {
    for (i = n; i > 0; i --) {
      p_dst[i - 1] = p_src[i - 1];
    }
  }
  return dst;
}

void *memcpy(void *out, const void *in, size_t n) {
  // panic("Not implemented");
  size_t i;
  unsigned char *p_out = (unsigned char *)out, *p_in = (unsigned char *)in;
  for (i = 0; i < n; i ++) {
    p_out[i] = p_in[i];
  }
  return out;
}

int memcmp(const void *s1, const void *s2, size_t n) {
  // panic("Not implemented");
  if (n == 0) return 0;

  size_t i;
  unsigned char *p_s1 = (unsigned char *)s1, *p_s2 = (unsigned char *)s2;
  for (i = 0; i < n; i ++) {
    if (p_s1[i] != p_s2[i]) return p_s1[i] - p_s2[i];
  }
  return 0;
}
```

此时运行 string 测试, 发现能够通过测试:

```
wzhixing@ubuntu-22:~/ics2026/am-kernels/tests/cpu-tests$ make ARCH=riscv32-nemu ALL=string run
# Building string-run [riscv32-nemu]
# Building am-archive [riscv32-nemu]
# Building klib-archive [riscv32-nemu]
+ OBJCOPY -> build/string-riscv32-nemu.bin
mainargs=
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/am-kernels/tests/cpu-tests/build/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/monitor/monitor.c:62 load_img] The image is /home/wzhixing/ics2026/am-kernels/tests/cpu-tests/build/string-riscv32-nemu.bin, st
ze = 1004
[src/monitor/monitor.c:28 welcome] Trace: ON
[src/monitor/monitor.c:29 welcome] If trace is enabled, a log file will be generated to record the trace. This may lead to a large l
og file. If it is not necessary, you can disable it in menuconfig
[src/monitor/monitor.c:32 welcome] Build time: 14:57:22, Oct 28 2026
Welcome to riscv32-NEMU!
For help, type "help"
[src/cpu/cpu-exec.c:156 cpu_exec] nemu: HIT GOOD TRAP at pc = 0x8000014c
[src/cpu/cpu-exec.c:121 statistic] host time spent = 27,999 us
[src/cpu/cpu-exec.c:122 statistic] total guest instructions = 3,341
[src/cpu/cpu-exec.c:123 statistic] simulation frequency = 119,325 inst/s
test list [1 item(s)]: string
[        string] PASS
```

IX. 实现 sprintf:

根据文件中的函数, 发现可以通过 vsprintf 统一处理 printf 和 sprintf:

```c
int sprintf(char *out, const char *fmt, ...) {
  // panic("Not implemented");
  out[0] = '\0';
  va_list args;
  va_start(args, fmt);
  int ret = vsprintf(out, fmt, args);
  va_end(args);
  return ret;
}
```

```c
int vsprintf(char *out, const char *fmt, va_list ap) {
  // panic("Not implemented");
  size_t i = 0, j, fmt_len = strlen(fmt);
  for (j = 0; j < fmt_len; ) {
    if (fmt[j] == '%') {
      j ++; int len = 1, width = 0;
      while (fmt[j] >= '0' && fmt[j] <= '9') {
        width = width * 10 + (fmt[j] - '0');
        len++, j++;
      }
      switch (fmt[j]) {
        case 'd': { // int
          int t = va_arg(ap, int);
          char str[64] = "";
          uint_to_str(t > 0 ? t : -t, str);
          if(t < 0) out[i ++] = '-';
          while (width > strlen(str) + (t < 0 ? 1 : 0)) out[i ++] = '0', width--;
          out[i] = '\0';
          strcat(out, str);
          i += strlen(str);
          break;
        }
        case 's': { // string
          char *s = va_arg(ap, char *);
          while (width > strlen(s)) out[i ++] = ' ', width--;
          out[i] = '\0';
          strcat(out, s);
          i += strlen(s);
          break;
        }
```

此时运行 hello-str 测试, 发现能够通过测试:

```
wzhixing@ubuntu-22:~/ics2026/am-kernels/tests/cpu-tests$ make ARCH=riscv32-nemu ALL=hello-str run
# Building hello-str-run [riscv32-nemu]
# Building am-archive [riscv32-nemu]
# Building klib-archive [riscv32-nemu]
+ OBJCOPY -> build/hello-str-riscv32-nemu.bin
mainargs=
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/am-kernels/tests/cpu-tests/build/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/monitor/monitor.c:62 load_img] The image is /home/wzhixing/ics2026/am-kernels/tests/cpu-tests/build/hello-str-riscv32-nemu.bin,
 size = 2736
[src/monitor/monitor.c:28 welcome] Trace: ON
[src/monitor/monitor.c:29 welcome] If trace is enabled, a log file will be generated to record the trace. This may lead to a large l
og file. If it is not necessary, you can disable it in menuconfig
[src/monitor/monitor.c:32 welcome] Build time: 14:57:22, Oct 28 2026
Welcome to riscv32-NEMU!
For help, type "help"
[src/cpu/cpu-exec.c:156 cpu_exec] nemu: HIT GOOD TRAP at pc = 0x8000010c
[src/cpu/cpu-exec.c:121 statistic] host time spent = 39,999 us
[src/cpu/cpu-exec.c:122 statistic] total guest instructions = 2,280
[src/cpu/cpu-exec.c:123 statistic] simulation frequency = 57,001 inst/s
test list [1 item(s)]: hello-str
[     hello-str] PASS
```

X. 实现 iringbuf:

使用一个字符串数组来存储最近的若干条指令, 并维护一个写指针 (每写一次自增 1, 并对 MAX_INST_TO_PRINT 取模):

```c
#ifdef CONFIG_ITRACE
static char ringbuf[MAX_INST_TO_PRINT][256];
static int  ringbuf_idx = 0;

static void iringbuf_write(const char* log) {
  strncpy(ringbuf[ringbuf_idx], log, 255);
  ringbuf[ringbuf_idx][255] = '\0';
  ringbuf_idx = (ringbuf_idx + 1) % MAX_INST_TO_PRINT;
}

static void iringbuf_print() {
  printf("Recent several instructions:\n");
  for (int i = 0; i < MAX_INST_TO_PRINT; i ++) {
    int k = (ringbuf_idx + i) % MAX_INST_TO_PRINT;
    if (ringbuf[k][0] != '\0')
      printf("        %s\n", ringbuf[k]);
  }
}
#endif
```

在 nemu/src/cpu/cpu-exec.c 的 `trace_and_difftest` 函数中, 利用先前反汇编的结果 (即 logbuf 中存储的内容) 写入 iringbuf 中:

```c
static void trace_and_difftest(Decode *_this, vaddr_t dnpc) {
#ifdef CONFIG_ITRACE_COND
  if (ITRACE_COND) { log_write("%s\n", _this->logbuf); }
#endif
  if (g_print_step) { IFDEF(CONFIG_ITRACE, puts(_this->logbuf)); }

#ifdef CONFIG_ITRACE
  iringbuf_write(_this->logbuf);
#endif

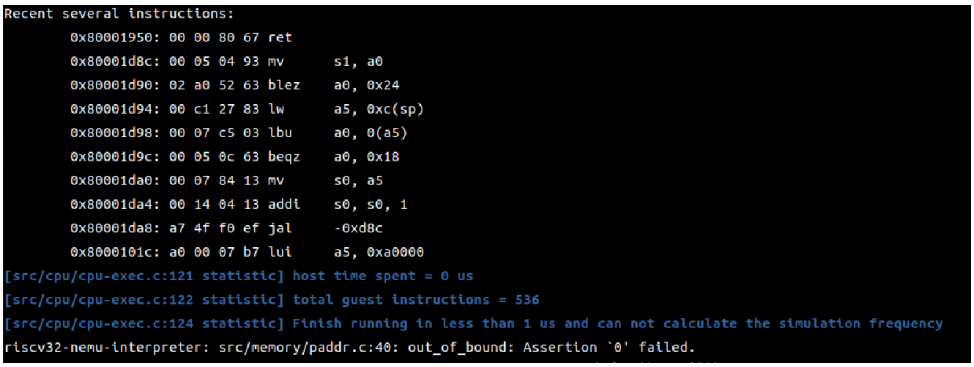
  IFDEF(CONFIG_DIFFTEST, difftest_step(_this->pc, dnpc));

#ifdef CONFIG_WATCHPOINT
  if (check_all_wp()) {
    printf("You hit a watchpoint.\n");
    if(nemu_state.state == NEMU_RUNNING) nemu_state.state = NEMU_STOP;
  }
#endif
}
```

当出现异常需要打印错误信息时, 即 cpu-exec.c 中的 `assert_fail_msg` 函数中, 我们把 iringbuf 中的信息打印出来:

```c
void assert_fail_msg() {
  isa_reg_display();
#ifdef CONFIG_ITRACE
  iringbuf_print();
#endif
  statistic();
}
```

一个示例:



XI. 实现 mtrace:

在 nemu/src/memory/paddr.c 中的内存读写接口函数记录访存踪迹, 同时在 Kconfig 中添加相应的选项 “mtrace”:

```c
word_t paddr_read(paddr_t addr, int len) {
  if (likely(in_pmem(addr))) {
    word_t res = pmem_read(addr, len);
#ifdef CONFIG_MTRACE
    printf("[mtrace] read  "FMT_PADDR"(%d Byte(s)) -> "FMT_WORD"\n", addr, len, res);
#endif
    return res;
  }
  IFDEF(CONFIG_DEVICE, return mmio_read(addr, len));
  out_of_bound(addr);
  return 0;
}

void paddr_write(paddr_t addr, int len, word_t data) {
  if (likely(in_pmem(addr))) {
#ifdef CONFIG_MTRACE
    printf("[mtrace] write "FMT_PADDR"(%d Byte(s)) <- "FMT_WORD"\n", addr, len, data);
#endif
    pmem_write(addr, len, data);
    return;
  }
  IFDEF(CONFIG_DEVICE, mmio_write(addr, len, data); return);
  out_of_bound(addr);
}
```

一个示例:

```
(nemu) c
[mtrace] read  0x80000000(4 Byte(s)) -> 0x00000297
[mtrace] read  0x80000004(4 Byte(s)) -> 0x00028823
[mtrace] write 0x80000010(1 Byte(s)) <- 0x00000000
[mtrace] read  0x80000008(4 Byte(s)) -> 0x0102c503
[mtrace] read  0x80000010(1 Byte(s)) -> 0x00000000
[mtrace] read  0x8000000c(4 Byte(s)) -> 0x00100073
```

XII. 实现 ftrace:

- 首先在 nemu/src/monitor/monitor.c 的 `parse_args` 函数中添加 “-f” 选项, 为 nemu 传入一个 elf 文件:

```
static int parse_args(int argc, char *argv[]) {
  const struct option table[] = {
    {"batch"    , no_argument      , NULL, 'b'},
    {"log"      , required_argument, NULL, 'l'},
    {"diff"     , required_argument, NULL, 'd'},
    {"ftrace"   , required_argument, NULL, 'f'},
    {"port"     , required_argument, NULL, 'p'},
    {"help"     , no_argument      , NULL, 'h'},
    {0          , 0                , NULL,  0 },
  };
  int o;
  while ( (o = getopt_long(argc, argv, "-bhl:d:f:p:", table, NULL)) != -1) {
    switch (o) {
      case 'b': sdb_set_batch_mode(); break;
      case 'p': sscanf(optarg, "%d", &difftest_port); break;
      case 'f': elf_file = optarg; break;
      case 'l': log_file = optarg; break;
      case 'd': diff_so_file = optarg; break;
      case 1: img_file = optarg; return 0;
      default:
        printf("Usage: %s [OPTION...] IMAGE [args]\n\n", argv[0]);
        printf("\t-b,--batch              run with batch mode\n");
        printf("\t-l,--log=FILE           output log to FILE\n");
        printf("\t-f,--ftrace=FILE        use ELF FILE for ftrace\n");
        printf("\t-d,--diff=REF_SO        run DiffTest with reference REF_SO\n");
        printf("\t-p,--port=PORT          run DiffTest with port PORT\n");
        printf("\n");
        exit(0);
    }
  }
  return 0;
}
```

- 接着在 nemu/src/utils/ftrace.c 和 nemu/include/utils.h 中对应部分添加相关函数原型和实现。

  先初始化 ftrace:

```
void init_ftrace(const char *elf_file) {
    Assert(elf_file != NULL, "[ftrace] ELF file cannot be NULL\n");

    Assert(parse_elf(elf_file), "[ftrace] Fail to load %s\n", elf_file);

    enabled = true;

    printf("[ftrace] Initialized successfully!\n");
    printf("[ftrace] loaded %d functions\n", nr_funct);
}
```

  其中 `parse_elf` 函数解析了该 elf 文件, 并从中获取了文件中的各个函数的信息, 包括函数名 name, 起始地址 start 和函数大小 size。

```
uint32_t shoff = rd32(elf_data + 32);
uint16_t shentsize = rd16(elf_data + 46);
uint16_t shnum = rd16(elf_data + 48);

for (int i = 0; i < shnum; i ++) {
    unsigned char *sh = elf_data + shoff + i * shentsize;

    if (rd32(sh + 4) != SHT_SYMTAB) continue;

    uint32_t sym_off = rd32(sh + 16);
    uint32_t sym_size = rd32(sh + 20);
    uint32_t str_idx = rd32(sh + 24);
    uint32_t sym_ent = rd32(sh + 36);
    if (sym_ent == 0) continue;

    unsigned char *strsh = elf_data + shoff + str_idx * shentsize;
    uint32_t str_off = rd32(strsh + 16);

    int n = sym_size / sym_ent;

    for (int j = 0; j < n && nr_funct < NR_FUNCT; j++) {
        unsigned char *sym = elf_data + sym_off + j * sym_ent;

        uint32_t nameoff = rd32(sym);
        uint32_t value   = rd32(sym + 4);
        uint32_t size    = rd32(sym + 8);
        unsigned char info = sym[12];

        if ((info & 0xf) != STT_FUNC) continue;
        if (size == 0) continue;

        funct[nr_funct].name  = (char *)elf_data + str_off + nameoff;
        funct[nr_funct].start = value;
        funct[nr_funct].size  = size;
        nr_funct++;
    }
```

- 最后, 实现 `ftrace_call` 和 `ftrace_ret` 函数,并在 nemu/src/isa/riscv32/inst.c 中的 jal 和 jalr 指令中根据目的寄存器 rd、立即数 imm 等来判断此时跳转代表的是 call 还是 ret。

```
void ftrace_call(vaddr_t pc, vaddr_t target) {
    if (!enabled) return;
    char *p = ftrace_buf;
    p += snprintf(p, sizeof(ftrace_buf), FMT_WORD ": %*scall [%s@" FMT_WORD "]",
        pc, depth * 2, "", find_funct(target), target);
    printf("%s\n", ftrace_buf);
    depth ++;
}

void ftrace_ret(vaddr_t pc) {
    if (!enabled) return;
    if (depth > 0) depth --;
    char *p = ftrace_buf;
    p += snprintf(p, sizeof(ftrace_buf), FMT_WORD ": %*sret  [%s]",
        pc, depth * 2, "", find_funct(pc));
    printf("%s\n", ftrace_buf);
}
```

```
INSTPAT("??????? ????? ????? ??? ????? 11011 11", jal    , J,
  R(rd) = s->pc + 4, s->dnpc = s->pc + imm
  #ifdef CONFIG_FTRACE
    ; if (rd == 1) ftrace_call(s->pc, s->dnpc)
  #endif
);
INSTPAT("??????? ????? ????? 000 ????? 11001 11", jalr   , I,
  R(rd) = s->pc + 4, s->dnpc = (src1 + imm) & (~1)
  #ifdef CONFIG_FTRACE
    ; if (rd == 0 && BITS(s->isa.inst, 19, 15) == 1 && imm == 0) ftrace_ret(s->pc);
    else if (rd == 1) ftrace_call(s->pc, s->dnpc)
  #endif
);
```

运行 recursion 后 ftrace 的部分输出:

```
Welcome to riscv32-NEMU!
For help, type "help"
0x8000000c: call [_trm_init@0x80000254]
0x80000264:   call [main@0x800001c8]
0x800001e8:     call [f0@0x80000010]
0x8000016c:       call [f2@0x800000a4]
0x800000f0:         call [f1@0x8000005c]
0x8000016c:           call [f2@0x800000a4]
0x800000f0:             call [f1@0x8000005c]
0x8000016c:               call [f2@0x800000a4]
0x800000f0:                 call [f1@0x8000005c]
0x8000016c:                   call [f2@0x800000a4]
0x800000f0:                     call [f1@0x8000005c]
0x8000016c:                       call [f2@0x800000a4]
0x800000f0:                         call [f1@0x8000005c]
0x8000016c:                           call [f2@0x800000a4]
0x80000058:                           ret  [f0]
0x80000100:                         ret  [f2]
0x80000180:                         call [f2@0x800000a4]
0x800000f0:                           call [f1@0x8000005c]
0x80000058:                           ret  [f0]
0x80000100:                         ret  [f2]
0x800001a8:                       ret  [f3]
0x80000100:                     ret  [f2]
0x80000180:                     call [f2@0x800000a4]
0x800000f0:                       call [f1@0x8000005c]
0x8000016c:                         call [f2@0x800000a4]
```

XIII. 运行 Hello World:

在 menuconfig 中打开 device 选项后, 到 am-kernels/kernels/hello 目录下运行 nemu。

```
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/am-kernels/kernels/hello/build/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'serial' at [0xa00003f8, 0xa00003ff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'rtc' at [0xa0000048, 0xa000004f]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vgactl' at [0xa0000100, 0xa0000107]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vmem' at [0xa1000000, 0xa10752ff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'keyboard' at [0xa0000060, 0xa0000063]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio' at [0xa0000200, 0xa0000217]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio-sbuf' at [0xa1200000, 0xa120ffff]
[src/monitor/monitor.c:62 load_img] The image is /home/wzhixing/ics2026/am-kernels/kernels/hello/build/hello-riscv32-nemu.bin, size
= 292
[src/monitor/monitor.c:28 welcome] Trace: OFF
[src/monitor/monitor.c:32 welcome] Build time: 21:07:51, Oct 28 2026
Welcome to riscv32-NEMU!
For help, type "help"
Hello, AbstractMachine!
mainargs = ''.
[src/cpu/cpu-exec.c:156 cpu_exec] nemu: HIT GOOD TRAP at pc = 0x800000b0
[src/cpu/cpu-exec.c:121 statistic] host time spent = 0 us
[src/cpu/cpu-exec.c:122 statistic] total guest instructions = 348
[src/cpu/cpu-exec.c:124 statistic] Finish running in less than 1 us and can not calculate the simulation frequency
```

XIV. 实现 printf:

利用之前已经实现好的 vsprintf 函数和 `putch` 函数逐个输出字符来实现 printf。

```c
int printf(const char *fmt, ...) {
  char tmp[MAX_READ_LEN];
  tmp[0] = '\0';
  va_list args;
  va_start(args, fmt);
  int ret = vsprintf(tmp, fmt, args);
  va_end(args);

  if (ret > 0) {
    for (const char *p = tmp; *p; p ++) putch(*p);
  }

  return ret;
}
```

XV. 运行 alu-tests:

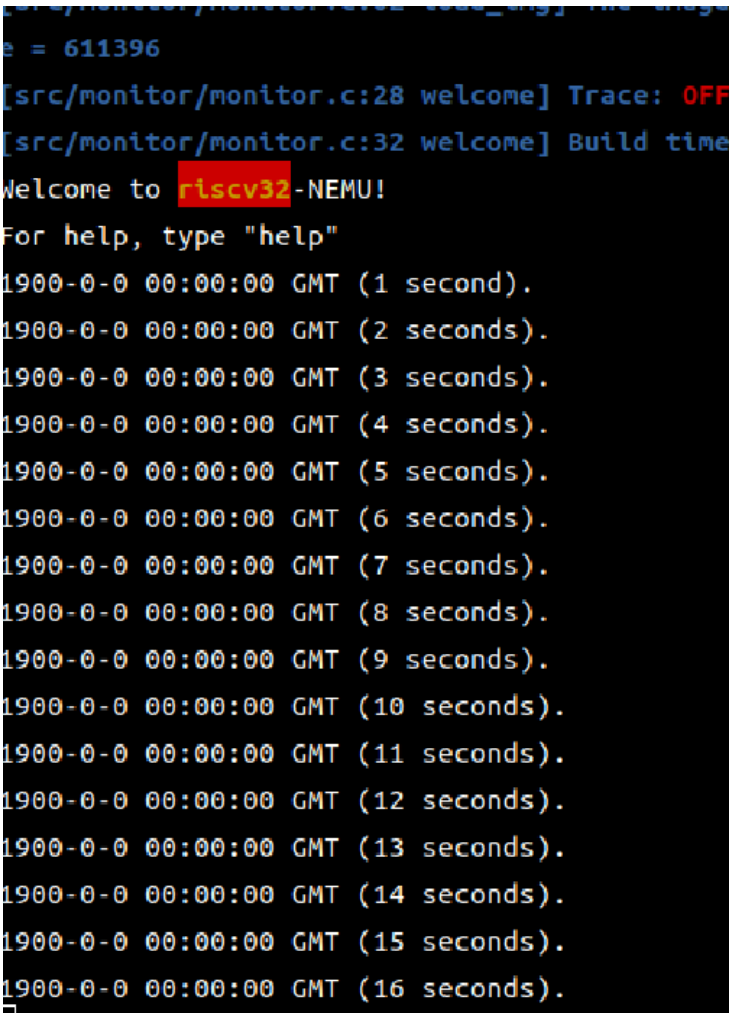
```
/home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpreter -b -l /home/wzhixing/ics2026/am-kernels/tests/alu-tests/build/nemu-log.tx
t  /home/wzhixing/ics2026/am-kernels/tests/alu-tests/build/alutest-riscv32-nemu.bin
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/am-kernels/tests/alu-tests/build/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'serial' at [0xa00003f8, 0xa00003ff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'rtc' at [0xa0000048, 0xa000004f]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vgactl' at [0xa0000100, 0xa0000107]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vmem' at [0xa1000000, 0xa10752ff]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'keyboard' at [0xa0000060, 0xa0000063]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio' at [0xa0000200, 0xa0000217]
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio-sbuf' at [0xa1200000, 0xa120ffff]
[src/monitor/monitor.c:62 load_img] The image is /home/wzhixing/ics2026/am-kernels/tests/alu-tests/build/alutest-riscv32-nemu.bin, s
ize = 967988
[src/monitor/monitor.c:28 welcome] Trace: OFF
[src/monitor/monitor.c:32 welcome] Build time: 21:07:51, Oct 28 2026
Welcome to riscv32-NEMU!
For help, type "help"
[src/cpu/cpu-exec.c:156 cpu_exec] nemu: HIT GOOD TRAP at pc = 0x800ebb34
[src/cpu/cpu-exec.c:121 statistic] host time spent = 0 us
[src/cpu/cpu-exec.c:122 statistic] total guest instructions = 69,855
[src/cpu/cpu-exec.c:124 statistic] Finish running in less than 1 us and can not calculate the simulation frequency
```

XVI. 实现 IOE:

在 abstract-machine/am/src/platform/nemu/ioe/timer.c 中根据手册和源代码完成 `__am_timer_uptime` 函数:

```c
void __am_timer_uptime(AM_TIMER_UPTIME_T *uptime) {
  uint32_t hi = inl(RTC_ADDR + 4);
  uint32_t lo = inl(RTC_ADDR);
  uptime->us = ((uint64_t)hi << 32) | lo;
}
```

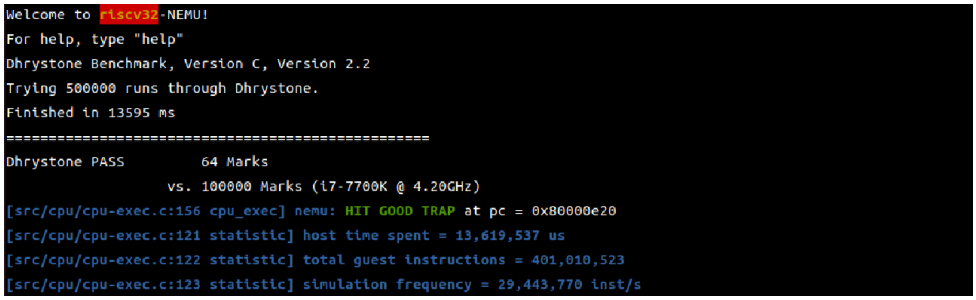
完成后进行 rtc 的测试:



XVII. 看看 NEMU 跑多快:

运行各个 benchmark:

- dhrystone:



- coremark:

```
Welcome to riscv32-NEMU!
For help, type "help"
Running CoreMark for 1000 iterations
2K performance run parameters for coremark.
CoreMark Size    : 666
Total time (ms)  : 10905
Iterations       : 1000
Compiler version : GCC11.4.0
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0xd340
Finised in 10905 ms.
==================================================
CoreMark PASS       267 Marks
               vs. 100000 Marks (i7-7700K @ 4.20GHz)
[src/cpu/cpu-exec.c:156 cpu_exec] nemu: HIT GOOD TRAP at pc = 0x80002384
[src/cpu/cpu-exec.c:121 statistic] host time spent = 10,907,252 us
[src/cpu/cpu-exec.c:122 statistic] total guest instructions = 305,935,563
[src/cpu/cpu-exec.c:123 statistic] simulation frequency = 28,048,821 inst/s
```

- microbench:

```
======= Running MicroBench [input *ref*] =======
[qsort] Quick sort: * Passed.
  min time: 1000.020 ms [440]
[queen] Queen placement: * Passed.
  min time: 1281.025 ms [317]
[bf] Brainf**k interpreter: * Passed.
  min time: 6894.135 ms [243]
[fib] Fibonacci number: * Passed.
  min time: 11228.211 ms [179]
[sieve] Eratosthenes sieve: * Passed.
  min time: 13977.247 ms [249]
[15pz] A* 15-puzzle search: * Passed.
  min time: 1423.025 ms [376]
[dinic] Dinic's maxflow algorithm: * Passed.
  min time: 1669.028 ms [490]
[lzip] Lzip compression: * Passed.
  min time: 1904.032 ms [356]
[ssort] Suffix sort: * Passed.
  min time: 644.011 ms [621]
[md5] MD5 digest: * Passed.
  min time: 10559.166 ms [143]
==================================================
MicroBench PASS        341 Marks
                  vs. 100000 Marks (i9-9900K @ 3.60GHz)
Scored time: 50579.900 ms
Total  time: 57653.019 ms
```

XVIII. 实现 dtrace:

在 nemu/src/device/io/mmio.c 中的两个总线接口函数 `mmio_read` 和 `mmio_write` 中记录访问设备的踪迹，这里采用了类似 mtrace 的设计:

```
/* bus interface */
word_t mmio_read(paddr_t addr, int len) {
  IOMap* cur_map = fetch_mmio_map(addr);
  word_t res = map_read(addr, len, cur_map);
#ifdef CONFIG_DTRACE
  printf("[dtrace] read  %s\t"FMT_PADDR" (%d Byte(s)) -> "FMT_WORD"\n", cur_map->name, addr, len, res);
#endif
  return res;
}

void mmio_write(paddr_t addr, int len, word_t data) {
  IOMap* cur_map = fetch_mmio_map(addr);
#ifdef CONFIG_DTRACE
  printf("[dtrace] write %s\t"FMT_PADDR" (%d Byte(s)) <- "FMT_WORD"\n", cur_map->name, addr, len, data);
#endif
  map_write(addr, len, data, cur_map);
}
```

同样地, 我们也在 Kconfig 中添加相应的选项。

XIX. 实现 IOE(2):

在 abstract-machine/am/src/platform/nemu/ioe/input.c 中的 `__am_input_keybrd` 函数中实现键盘输入的功能:

```
#define KEYDOWN_MASK 0x8000

void __am_input_keybrd(AM_INPUT_KEYBRD_T *kbd) {
  int32_t cur_keycode = inl(KBD_ADDR);
  kbd->keydown = KEYDOWN_MASK & cur_keycode;
  kbd->keycode = ~KEYDOWN_MASK & cur_keycode;
}
```

完成后进行 keyboard 测试:

```
Welcome to riscv32-NEMU!
For help, type "help"
Try to press any key (uart or keyboard)...
Got  (kbd): Q (29) DOWN
Got  (kbd): Q (29) UP
Got  (kbd): W (30) DOWN
Got  (kbd): W (30) UP
Got  (kbd): E (31) DOWN
Got  (kbd): E (31) UP
Got  (kbd): R (32) DOWN
Got  (kbd): R (32) UP
Got  (kbd): T (33) DOWN
Got  (kbd): T (33) UP
Got  (kbd): Y (34) DOWN
Got  (kbd): Y (34) UP
Got  (kbd): 1 (15) DOWN
Got  (kbd): 1 (15) UP
Got  (kbd): 2 (16) DOWN
Got  (kbd): 2 (16) UP
Got  (kbd): 3 (17) DOWN
Got  (kbd): 3 (17) UP
```

XX. 实现 IOE(3):

- 屏幕大小:

  在 nemu/src/device/vga.c 中的 `init_vga` 函数中, vgactl_port_base 申请了 8 字节 = 2 个 4 字节的空间, 且 vgactl_port_base[0] 中存储了屏幕的宽和高。

```
void init_vga() {
  vgactl_port_base = (uint32_t *)new_space(8);
  vgactl_port_base[0] = (screen_width() << 16) | screen_height();
```

进而在 abstract-machine/am/src/platform/nemu/ioe/gpu.c 中的 `__am_gpu_config` 函数依据上图中的信息进行屏幕宽和高的配置:

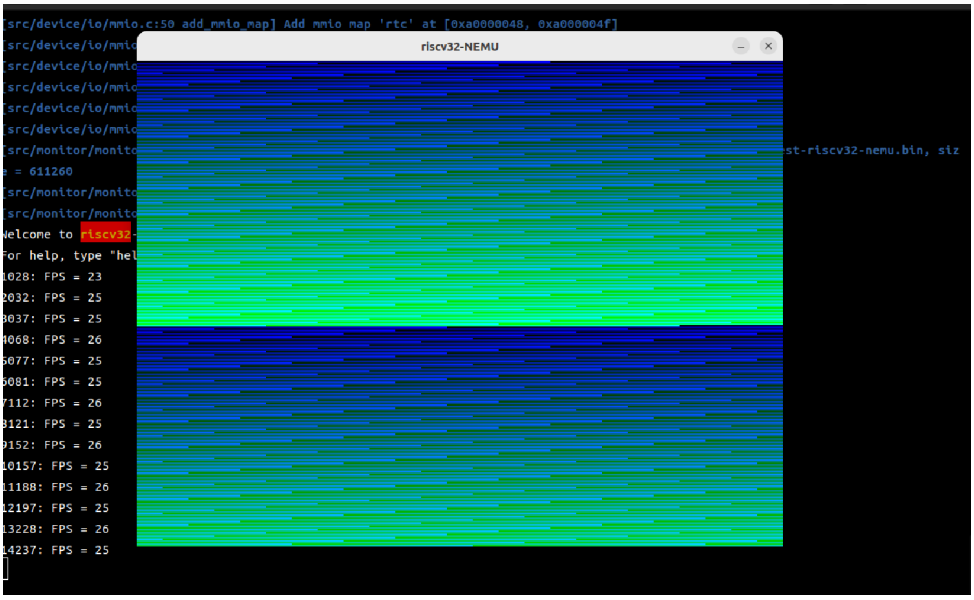
```
void __am_gpu_config(AM_GPU_CONFIG_T *cfg) {
  *cfg = (AM_GPU_CONFIG_T) {
    .present = true, .has_accel = false,
    .width = 0, .height = 0,
    .vmemsz = 0
  };
  int32_t hw  = inl(VGACTL_ADDR);
  cfg->height = hw & 0xffff;
  cfg->width  = hw >> 16;
}
```

- 同步屏幕:

  根据 `init_vga` 函数中申请的 8 字节空间可知, vgactl_port_base[1] 应为屏幕同步寄存器, 因此在 nemu/src/device/vga.c 的 `vga_update_screen` 函数中利用 vgactl_port_base[1] 来实现屏幕刷新。

```
void vga_update_screen() {
  // TODO: call `update_screen()` when the sync register is non-zero,
  // then zero out the sync register
  if (vgactl_port_base[1]) {
    update_screen();
    vgactl_port_base[1] = 0;
  }
}
```

实现以后, 进行 display_test 测试:



XXI. 实现 IOE(4):

根据手册中的信息和已给出的源代码实现 abstract-machine/am/src/platform/nemu/ioe/gpu.c 中的 `__am_gpu_fbdraw` 函数。

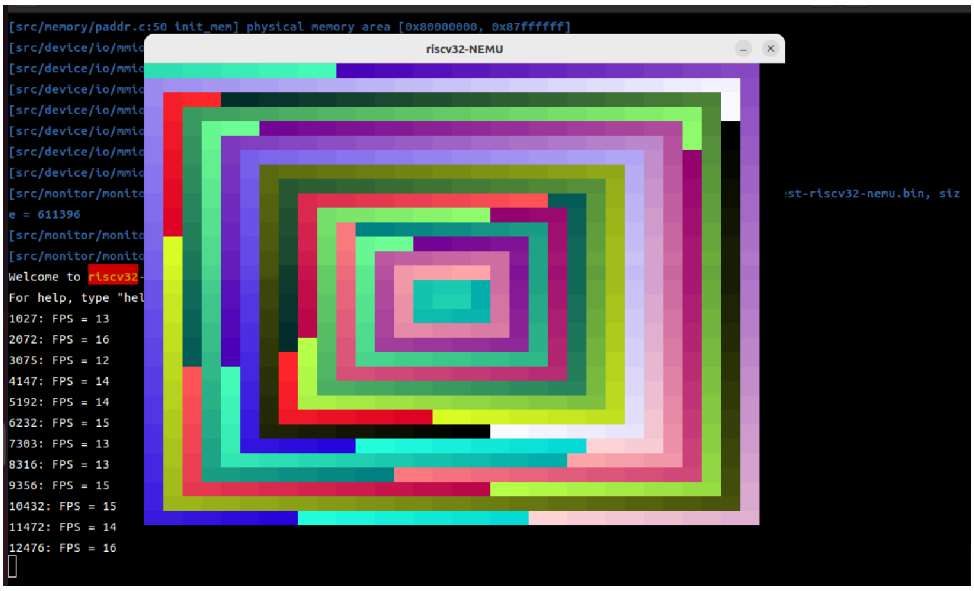
```
void __am_gpu_fbdraw(AM_GPU_FBDRAW_T *ctl) {
  int x = ctl->x, y = ctl->y;
  int h = ctl->h, w = ctl->w;

  uint32_t *fb = (uint32_t *)(uintptr_t)FB_ADDR;
  uint32_t *buf = ctl->pixels;
  AM_GPU_CONFIG_T cfg;
  ioe_read(AM_GPU_CONFIG, &cfg);

  for (int i = 0; i < h; i ++) {
    for (int j = 0; j < w; j ++) {
      fb[(y + i) * cfg.width + (x + j)] = buf[i * w + j];
    }
  }

  if (ctl->sync) {
    outl(SYNC_ADDR, 1);
  }
}
```

实现后重新进行 display_test 测试:

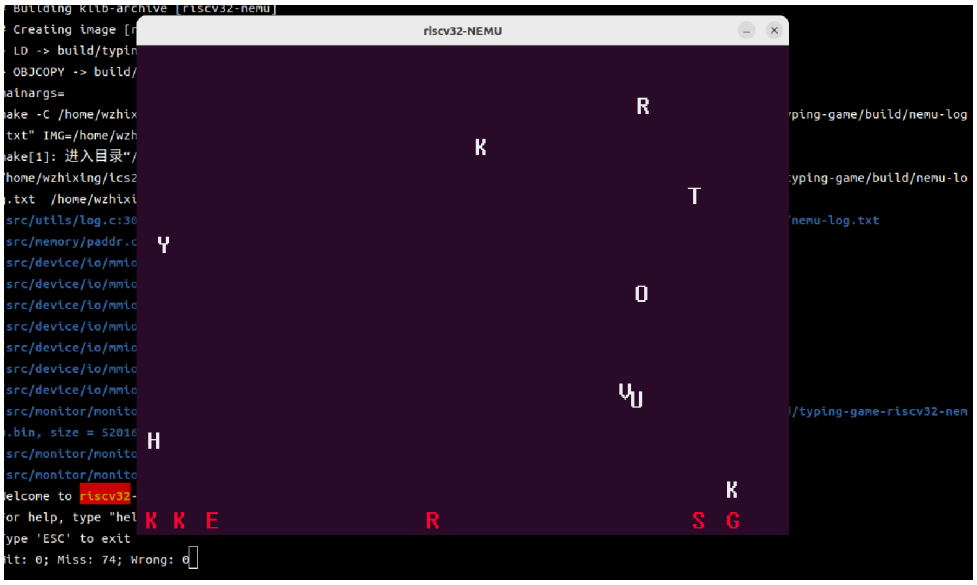


XXII. 展示你的计算机系统:

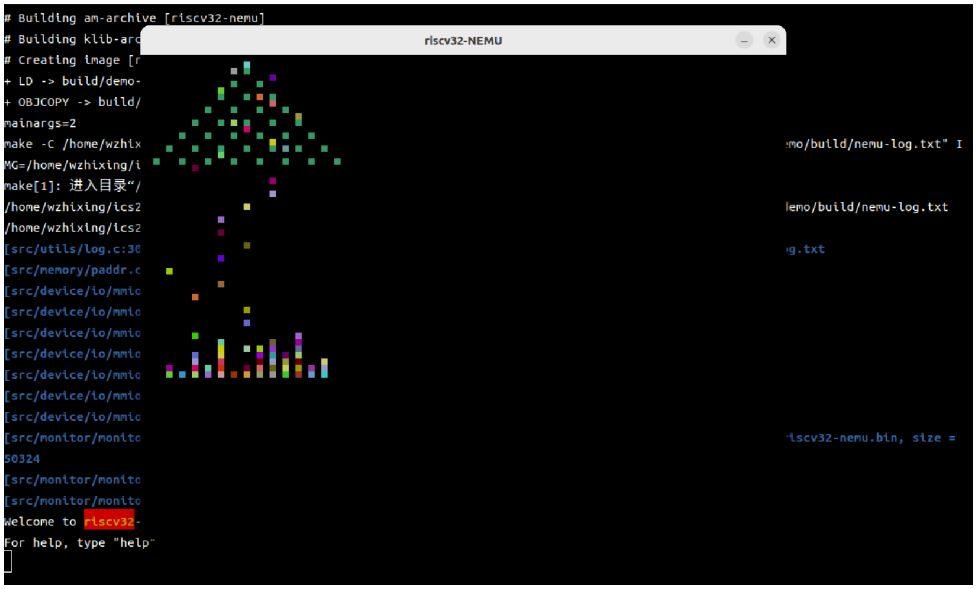
- 幻灯片播放:



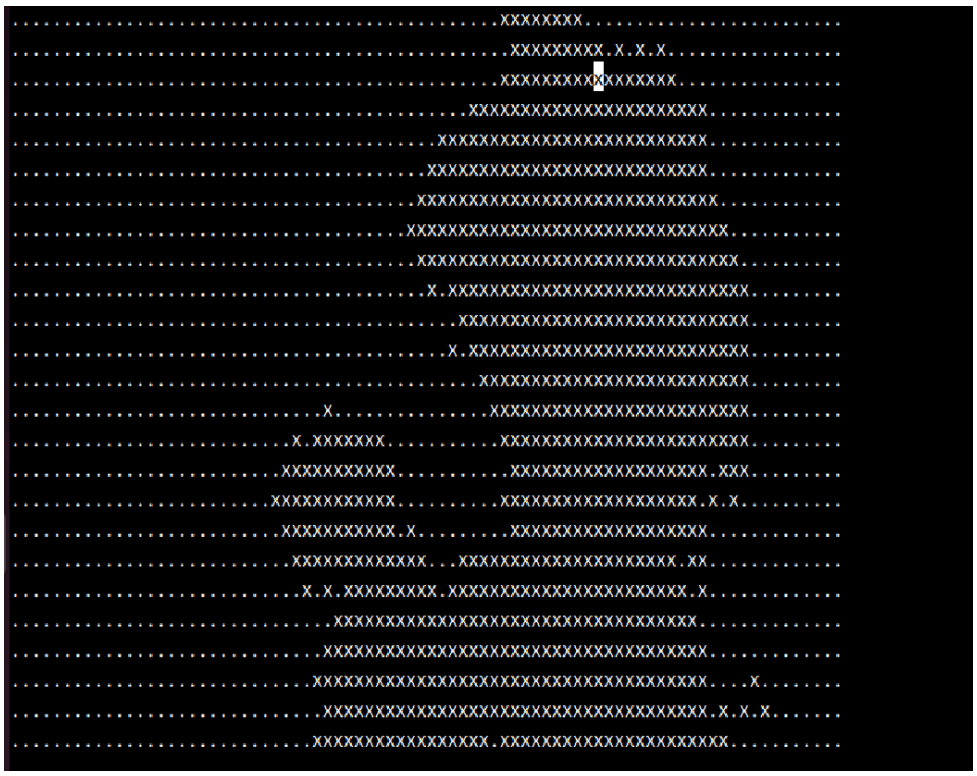
- 打字小游戏:



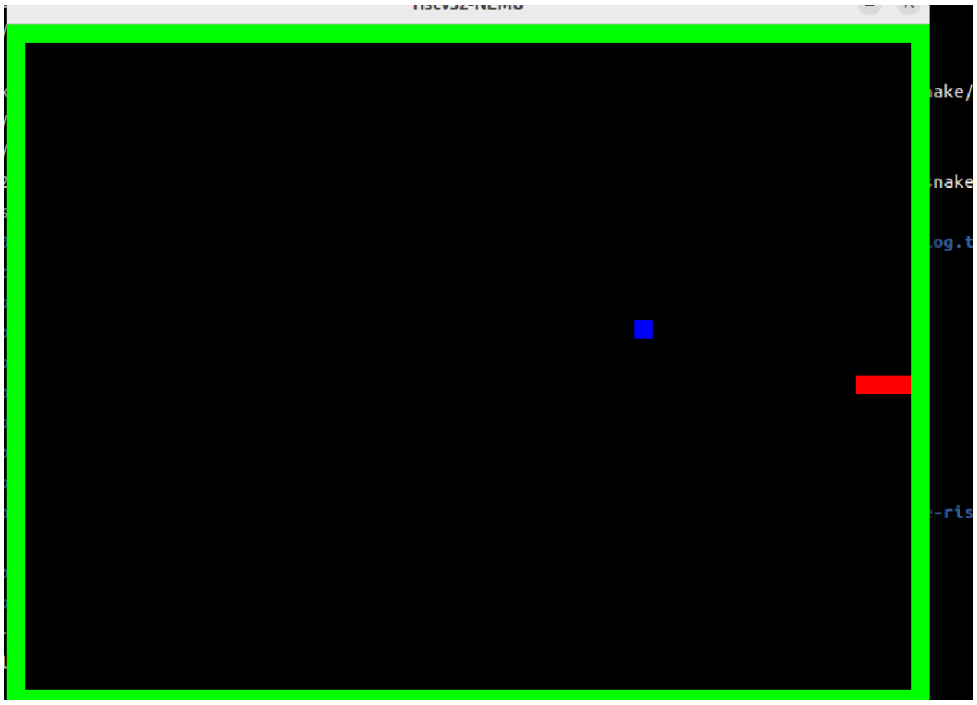
- 演示程序集合 (以高尔顿钉板为例):



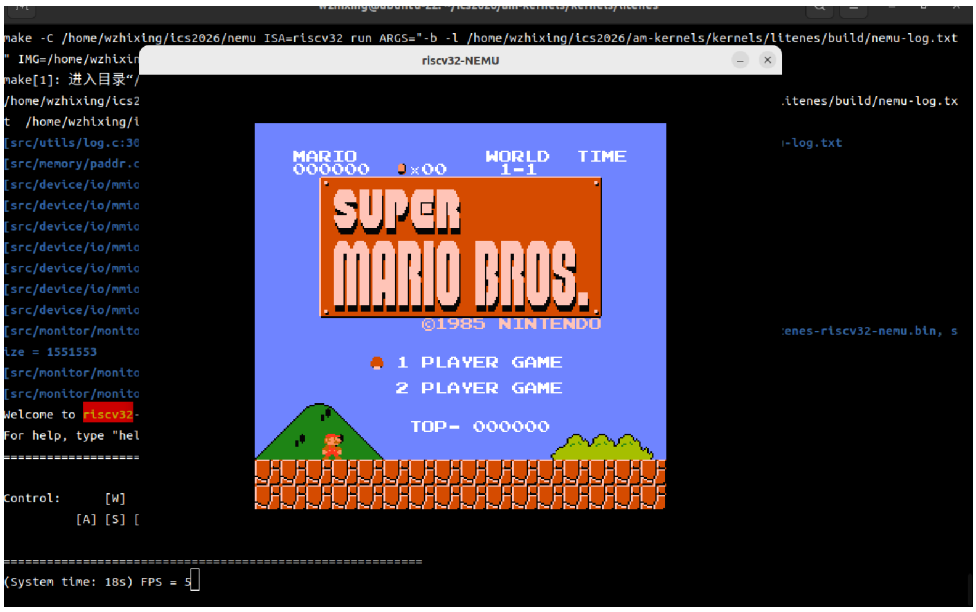
- “Bad Apple!!” PV 的播放:



- 贪吃蛇:



- 简单的红白机模拟器 LiteNES:



- 完整的红白机模拟器 FCEUX:



XXIII. 游戏是如何运行的:

打字小游戏运行时，当我们按下一个字母并命中时，首先 nemu 会接收到键盘码，而后 abstract-machine 通过 `ioe_read` 读取键盘控制器信息，并依据此信息调用函数 `check_hit`;

在 `check_hit` 函数中，由于我们命中屏幕上的某个字母，那么根据函数的运行逻辑，该函数会将距离屏幕底部最近的那个相匹配的字母的运动方向改变，即变为沿着屏幕底部往屏幕顶端移动，同时命中字符的总次数增加 1。

接下来，调用完 `check_hit` 函数返回 `main` 函数后，根据运行逻辑，应当调用 `render` 函数来生成画面。

在 `render` 函数中，我们首先将上一帧屏幕清屏，再根据当前屏幕上的字符的状态

使用 `__am_gpu_fbdraw` 函数进行绘制, 并在绘制完成后统一刷新屏幕。最后在终端上输出当前的命中次数、未击中次数和错误次数。

XXIV. 必答题:

- 程序是个状态机: 此问题在前文已回答;
- RTFSC: 此问题在前文已回答;
- 程序如何运行: 此问题在前文已回答;
- 编译与链接:

  对于在 nemu/include/cpu/ifetch.h 中由 static inline 开头定义的 `inst_fetch` 函数而言:

  – 只去掉 inline 不会发生报错;

  – 只去掉 static 在编译优化选项为`-O2` 时不会报错, 在`-O0` 时会报错, 且报错信息如下:

```
/usr/bin/ld: /home/wzhixing/ics2026/nemu/build/obj-riscv32-nemu-interpreter/src/isa/riscv32/inst.o: in function `isa_exec_once':
inst.c:(.text+0xff1dd): undefined reference to `inst_fetch'
/usr/bin/ld: /home/wzhixing/ics2026/nemu/build/obj-riscv32-nemu-interpreter/src/engine/interpreter/hostcall.o: in function `invalid_
inst':
hostcall.c:(.text+0x78): undefined reference to `inst_fetch'
/usr/bin/ld: hostcall.c:(.text+0x8c): undefined reference to `inst_fetch'
collect2: error: ld returned 1 exit status
```

  – 两者都去掉后一定会报错，且报错信息如下:

```
wzhixing@ubuntu-22:~/ics2026/nemu$ make
+ CC src/isa/riscv32/inst.c
+ CC src/engine/interpreter/hostcall.c
+ LD /home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpreter
/usr/bin/ld: /home/wzhixing/ics2026/nemu/build/obj-riscv32-nemu-interpreter/src/engine/interpreter/hostcall.o: in function `inst_fet
ch':
hostcall.c:(.text+0x0): multiple definition of `inst_fetch'; /home/wzhixing/ics2026/nemu/build/obj-riscv32-nemu-interpreter/src/isa/
riscv32/inst.o:inst.c:(.text+0x1780): first defined here
collect2: error: ld returned 1 exit status
make: *** [/home/wzhixing/ics2026/nemu/scripts/build.mk:54: /home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpreter] 错误 1
```

  阅读报错信息可知, 出现错误的原因是该函数出现了多重定义。

  – 分析添加 static/inline 后为何错误不会发生:

    1) static: 未添加 static 关键字时, `inst_fetch` 为全局强符号, 在每个包含它的文件中都会生成一个副本, 进而使链接器报 “重复定义” 的错误; 添加 static 关键字后, 该函数不再是全局强符号, 而是每个翻译单元各自保存一个 local 的私有副本, 因此不报错;

    2) inline: 在`-O2` 优化下, inline 关键字告诉编译器在调用该函数时直接对函数体进行展开, 不生成符号; 需要注意的是, 在`-O0` 选项下编译器并不一定会做出实际的内联优化, 而普通 inline 不提供外部定义，没展开时就可能会导致 undefined reference。

- 编译与链接:

  1. 添加一行 “volatile static int dummy” 后, 在我当前的配置下, nemu 中含有 34 个 dummy 变量的实体。

方法如图所示:

```
wzhixing@ubuntu-22:~/ics2026/nemu$ echo "symbol entities:"; readelf -sW build/riscv32-nemu-interpreter | awk '$4=="OBJECT" && $NF -
/^dummy(\.lto_priv\.[0-9]+)?$/ {n++} END{print n+0}'
symbol entities:
34
```

2. 接着在 nemu/include/debug.h 中添加一行 “volatile static int dummy”, dummy 实体数不变, 仍为 34 个。这是因为它们均是处于同一文件作用域下的同名符号的暂定定义 (tentative definition), 两者将发生合并成为同一个对象。

3. 当我们为两处 dummy 初始化后, 会出现如下问题:

```
wzhixing@ubuntu-22:~/ics2026/nemu$ make
+ CC src/nemu-main.c
In file included from src/nemu-main.c:16:
/home/wzhixing/ics2026/nemu/include/common.h:49:21: error: redefinition of ‘dumm
y’
   49 | volatile static int dummy = 0;
      |                     ^~~~~
In file included from /home/wzhixing/ics2026/nemu/include/common.h:47,
                 from src/nemu-main.c:16:
/home/wzhixing/ics2026/nemu/include/debug.h:43:21: note: previous definition of
‘dummy’ with type ‘int’
   43 | volatile static int dummy = 0;
      |                     ^~~~~
make: *** [/home/wzhixing/ics2026/nemu/scripts/build.mk:34：/home/wzhixing/ics20
26/nemu/build/obj-riscv32-nemu-interpreter/src/nemu-main.o] 错误 1
```

之前两处都没有初始化，都是同一文件作用域下的暂定定义。如上一问所述, C 允许同一个翻译单元中出现多个同名暂定定义，它们会合并为同一个对象，因此不会报错。

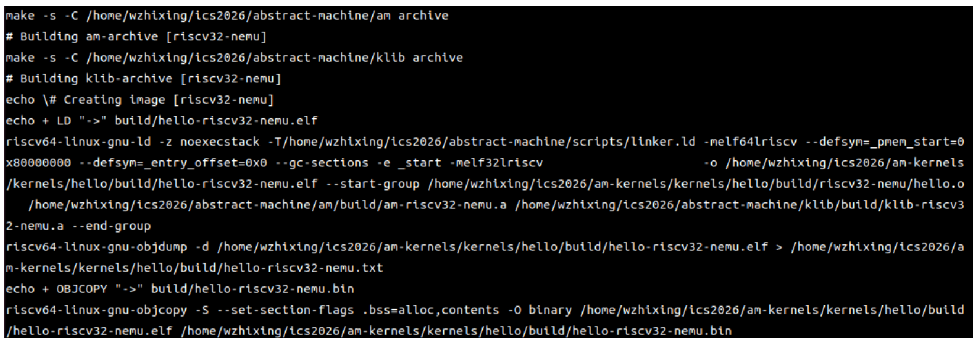
当两处都改为 “volatile static int dummy = 0” 后，每个带初始化的声明都是完整定义, 而不再是暂定定义。同一个翻译单元、同一文件作用域内, 同一个对象只能有一个完整定义，因此编译器报 redefinition of ‘dummy’。

- 了解 Makefile:

由于我实现的 nemu 是 riscv32-nemu, 那么就以 ARCH=riscv32-nemu 为例, 来分析 make 的行为。

当我们输入 “make ARCH=riscv32-nemu” 以后, 整个构建目标文件的过程是:

```
wzhixing@ubuntu-22:~/ics2026/am-kernels/kernels/hello$ make ARCH=riscv32-nemu -n
# Building hello-image [riscv32-nemu]
mkdir -p /home/wzhixing/ics2026/am-kernels/kernels/hello/build/riscv32-nemu/ && echo + CC hello.c
riscv64-linux-gnu-gcc -std=gnu11 -O2 -MMD -Wall -Werror -I/home/wzhixing/ics2026/am-kernels/kernels/hello/include -I/home/wzhixing/i
cs2026/abstract-machine/am/include/ -I/home/wzhixing/ics2026/abstract-machine/klib/include/ -D__ISA__=\"riscv32\" -D__ISA_RISCV32__
-D__ARCH__=riscv32-nemu -D__ARCH_RISCV32_NEMU -D__PLATFORM__=nemu -D__PLATFORM_NEMU -DARCH_H=\"arch/riscv.h\" -fno-asynchronous-unwi
nd-tables -fno-builtin -fno-stack-protector -Wno-main -U_FORTIFY_SOURCE -fvisibility=hidden -fno-pic -march=rv64g -mcmodel=medany -m
strict-align -march=rv32im_zicsr -mabi=ilp32    -static -fdata-sections -ffunction-sections -I/home/wzhixing/ics2026/abstract-machin
e/am/src/platform/nemu/include -DMAINARGS_MAX_LEN=64 -DMAINARGS_PLACEHOLDER=the_insert-arg_rule_in_Makefile_will_insert_mainargs_her
e -DISA_H=\"riscv/riscv.h\" -c -o /home/wzhixing/ics2026/am-kernels/kernels/hello/build/riscv32-nemu/hello.o /home/wzhixing/ics2026/
am-kernels/kernels/hello/hello.c
```



1) 第一步是先编译 hello.c 生成 hello.o 文件, 在后面的步骤中它将与其他文件链接;
2) 第二步是建立依赖库, 包括 am archive 和 klib archive;
3) 第三步是链接前两步生成的所有文件, 包括 hello.o 和两个链接库 am.a 与 klib.a，生成最终的 hello-riscv32-nemu.elf 文件;
4) 第四步是生成反汇编文件, 把上一步生成的 elf 文件 “objdump -d” 以后的结果写入 hello-riscv32-nemu.txt 中;
5) 最后一步是生成镜像文件, 利用 objcopy 生成 hello-riscv32-nemu.bin 文件供 nemu 使用。

具体地, hello 目录下的 Makefile 只设置了 NAME 和 SRCS，然后 include $(AM_HOME)/Makefile，实际的编译和链接规则在 abstract-machine 项目的 Makefile 中。再根据 abstract-machine 的 Makefile 可知, .c 文件会通过编译规则生成一个相对应的.o 文件; .h 文件不单独编译，而是通过 “#include” 展开，-I 指定头文件搜索路径。-MMD 参数让编译器生成.d 依赖文件，Makefile 再用 -include 读入这些.d，从而在头文件变化时重新编译相关目标。.a 文件则是进入子目录中利用子目录的 make 文件生成。最终, LINKAGE 中包括了 SRCS 生成的.o 文件和所需的库文件.a, 通过 ld 将各个文件链接生成最终的 elf 文件。

3 实验心得和想法

本部分实验难度增大, 需要阅读更多源代码, 了解更多编译、链接等计算机系统的知识。在这个过程中, 我对计算机系统的整体认识也逐渐加深。

完成整个 PA2 的过程让我收获到更多有用的知识，也体会到制作项目的不容易。在 PA2 的最后, 我们成功构建出一台简陋的 “计算机”，虽然它的性能较为低下，但是实现它的整个过程对我来说是一次新奇的体验。